

A CrewAI-Based Multi-Agent Framework for Evidence-Grounded Automated Code Review

Hakan Boğan*, İbrahim Arıkan*, Emrah İnan*

*Department of Computer Engineering, IZTECH

Email: hakanbogan@iyte.edu.tr, ibrahimarikan@iyte.edu.tr, emrahinan@iyte.edu.tr

Abstract—Large Language Models (LLMs) are increasingly integrated into software engineering workflows; however, their use in automated code review remains fundamentally limited by hallucination, lack of evidence grounding, and inconsistent reasoning across complex changes. Existing hybrid approaches that combine LLMs with static analyzers improve precision but do not provide structured role specialization, multi-step reasoning, or enterprise-level quality control. To address these limitations, this study proposes a CrewAI-based multi-agent code review framework composed of five specialized agents: Change Context Analyst, Security Reviewer, Style & Format Reviewer, Revision Proposer and Supervisor. Unlike prior work, our system enforces a strict evidence-based review paradigm in which every finding must be supported by static analysis outputs (Semgrep, Bandit, Ruff, ESLint), file-line references, and relevant code snippets. We empirically evaluate three operational modes—single-agent, multi-agent, and tools-only—across diverse pull request scenarios. Results show that the proposed multi-agent architecture significantly improves actionable review quality, reduces false positives, and enhances consistency. The study demonstrates that combining MAS principles, LLM orchestration, and static analysis grounding yields a more reliable, explainable and production-ready approach to automated code review than existing methods.

Index Terms—Multi-Agent Systems, Large Language Models, CrewAI, Automated Code Review, Static Analysis, Software Engineering, Evidence-Based Reasoning

I. INTRODUCTION

Code review is a fundamental practice in modern software development, supporting defect detection, maintainability improvement, and collaborative knowledge sharing across teams. Empirical evidence shows that effective code review can significantly reduce defect rates and improve long-term software quality [1], [2]. However, as the scale and frequency of pull requests (PRs) grow, manual review becomes a time-consuming bottleneck that imposes substantial cognitive and operational load on developers.

The emergence of Large Language Models (LLMs) has inspired new approaches to automating parts of the review workflow. Existing studies explore the use of LLMs for generating review comments, detecting bugs, and suggesting code improvements [3], [4]. Despite promising early results, most LLM-based systems rely on a single monolithic agent, which leads to several fundamental limitations. First, LLM-generated comments often lack actionability, providing abstract suggestions without concrete patches or remediation steps. Second, single-agent systems frequently produce high noise-to-signal ratios due to false positives or low-priority findings (“nits”),

reducing developers’ trust and overwhelming reviewers [5]. Third, coverage remains incomplete, as effective review requires expertise across multiple dimensions—security, logic, style, architecture—which a single agent seldom handles consistently. Moreover, LLM-based review systems commonly lack evidence grounding, generating speculative findings unsupported by static analysis tools or explicit code references [6], [7]. In practice, these limitations manifest along four concrete dimensions:

- 1) **Lack of actionability:** Many automated review systems produce comments that are technically correct but not directly actionable. They flag issues without proposing concrete patches, remediation steps, or prioritization, forcing developers to invest additional effort to translate feedback into changes.
- 2) **High noise-to-signal ratio:** Without sophisticated filtering, single-agent systems often generate a large volume of low-priority findings (“nits”) and false positives. This noise can overwhelm reviewers, induce alert fatigue, and ultimately reduce trust in automated review systems.
- 3) **Incomplete coverage:** A single monolithic agent struggles to thoroughly reason about security, logic, style, performance, and architecture at once. As a result, critical issues may be missed or analyzed only superficially, especially in complex pull requests.
- 4) **Insufficient evidence grounding:** Many systems rely almost exclusively on LLM inference without tying findings to static-analysis outputs, test coverage, or concrete code metrics. This lack of grounding makes it difficult for developers to validate findings and undermines the reliability of automated review.

Multi-Agent Systems (MAS) have recently demonstrated strong potential for improving reasoning, task decomposition, and coordination in various software engineering domains [8]–[11]. Modern LLM-driven MAS frameworks such as AutoGen [12], AgentVerse [13], CAMEL [14] and MetaGPT [15] highlight that role-specialized LLM agents can outperform single-agent models on tasks requiring structured reasoning. However, the application of MAS to automated code review remains underexplored. Existing multi-agent approaches lack unified evidence requirements, supervisor-level consolidation, or deep integration with static analysis tools [29], [30]. Concretely, there is a need for:

- **Evidence-grounded multi-agent architectures** that re-

quire every finding to be supported either by static-analysis outputs or explicit file–line references.

- **Coordinated teams of specialized agents** (e.g., security, style, logic, tests) that operate under a supervisor to reconcile conflicts and control quality.
- **Comprehensive evaluation frameworks** that measure actionability, noise reduction, and coverage using statistically rigorous metrics.
- **Fully reproducible experimental designs** that enable fair comparison between single-agent and multi-agent approaches.

To address these gaps, we propose a **CrewAI-based Multi-Agent Code Review Framework** that orchestrates five specialized agents operating in parallel: a Change & Context Analyst, a Security Reviewer, a Style & Format Reviewer, a Revision Proposer, and a Supervisor. A key innovation of the framework is an **evidence-based policy** requiring all findings to be grounded either in static analysis tool outputs (Semgrep, Bandit, Ruff, ESLint) or in explicit file–line references supported by code snippets. The Supervisor agent unifies all findings through deduplication, conflict resolution, severity calibration, and noise filtering, enabling a more reliable and consistent review process. In the extended project version of the framework, we further generalize this architecture to a richer team of specialized agents:

- **Change & Context Analyst (CCA)**: Analyzes pull-request diffs, validates alignment with the stated intent, and checks high-level logical consistency of the change set.
- **Security Reviewer (SR)**: Integrates Semgrep and Bandit outputs with LLM reasoning to detect both known vulnerability patterns and emerging security concerns.
- **Style & Format Reviewer (SFR)**: Consolidates linting and formatting issues from Ruff and ESLint while prioritizing style violations that materially affect readability and maintainability.
- **Logic Bug Reviewer (LR)**: Focuses on logical errors, control-flow problems, and algorithmic defects that may not be captured by static analyzers.
- **Performance Reviewer (PR)**: Identifies performance antipatterns, inefficient algorithms, and unnecessary complexity that could degrade runtime behavior.
- **Documentation Reviewer (DR)**: Evaluates the completeness and clarity of documentation, comments, and API contracts associated with the change.
- **Test Coverage Reviewer (TR)**: Assesses whether the modified code is sufficiently covered by tests and whether new tests are needed.

The **Revision Proposer (RP)** generates actionable patches or detailed remediation guidance for issues that can be automatically fixed, while the **Supervisor** enforces evidence requirements, removes duplicates, resolves conflicts between agents, and applies configurable noise-reduction strategies.

This work evaluates whether such an evidence-grounded multi-agent framework can outperform single-agent and tools-

only baselines in three dimensions: actionability, noise reduction, and coverage.

(Actionability Hypothesis): The tool-integrated multi-agent code review system achieves a higher actionability rate than the single-agent baseline, as measured by the proportion of findings that include concrete patches or clear, high-severity guidance with precise locations.

(Noise Reduction Hypothesis): The multi-agent system produces a lower noise rate than the single-agent baseline, as measured by the proportion of false positives among all findings when compared against expert-labeled ground truth.

(Coverage Hypothesis): The multi-agent system provides better coverage of important issues than the single-agent baseline, as measured by the fraction of expert-labeled critical issues that are successfully detected by the automated system.

Our experimental design includes real-world pull requests, expert ground-truth labels, reproducible configurations, and statistical significance testing. The contributions of this work are as follows:

- 1) **New evidence-grounded multi-agent architecture**: We present a CrewAI-based multi-agent code review framework that coordinates specialized agents through a supervisor pattern and demonstrate how agent specialization can improve review quality.
- 2) **Evidence requirement framework**: We introduce a systematic policy that requires every finding to be grounded in static-analysis tool outputs or explicit file–line references, which significantly reduces speculative suggestions and increases developer trust.
- 3) **Comprehensive evaluation methodology**: We design a rigorous evaluation framework with clearly defined metrics (actionability, noise, coverage, performance), statistical tests, effect-size analysis, and inter-rater reliability checks, providing a solid basis for future work in automated code review.
- 4) **Open-source implementation**: We release a production-ready implementation of the framework with detailed documentation, enabling independent reproduction of our experiments and serving as a foundation for follow-up research.
- 5) **Empirical validation**: We present controlled experiments on real-world pull requests that compare single-agent, multi-agent, and tools-only modes, providing empirical evidence that the proposed multi-agent system improves actionability, reduces noise, and enhances coverage.

II. RELATED WORK

Large Language Model (LLM) based autonomous multi-agent systems (MAS) have emerged as a powerful paradigm for solving complex tasks. Foundational agent models such as the BDI architecture by Jennings and Wooldridge established

rational decision-making principles for multi-agent collaboration [8], while Panait and Luke provided a comprehensive survey on cooperative multi-agent learning challenges [9]. Team-based agent architectures [10] and distributed reinforcement learning approaches such as independent Q-learning [11] further advanced early MAS research.

With the advent of LLMs, MAS research experienced a significant shift. Frameworks such as AutoGen [12], AgentVerse [13], and CAMEL [14] demonstrated that multiple role-specialized LLM agents outperform single LLM systems on complex reasoning and coding tasks. MetaGPT introduced standardized operating procedures (SOPs) into multi-agent workflows, enabling structured software engineering pipelines [15], whereas ChatDev applied communicative agents to software development but lacked consistency in structured outputs [16].

More recent efforts focused on dynamic and adaptive multi-agent architectures. Aime proposed evolving plans and real-time feedback [17], while LbMAS introduced blackboard-driven collaborative reasoning [18]. Orchestrator leveraged active inference for agent coordination under uncertainty [19]. These works indicate a shift from static workflows to context-aware and adaptive MAS.

The expansion of MAS ecosystems motivated standardization through protocols such as Model Context Protocol (MCP) [20], Agent Network Protocol (ANP) [21], and Google’s Agent2Agent protocol [22], while Agora presented a meta-protocol enabling natural-language-driven coordination [23].

In software engineering, benchmarks such as HumanEval and MBPP evaluate code generation ability, where MetaGPT achieved state-of-the-art performance [15]. More advanced evaluations such as SWE-Bench [24], SWE-Bench PRO [25], and the Routine framework [26] assess multi-file, long-horizon agent reasoning and tool-augmented workflows.

LLM-based code review research has shown that single-agent models suffer from hallucinations, weak context reasoning, and inconsistent findings [5]–[7]. Hybrid approaches integrate static analyzers with LLMs but lack role specialization and systematic quality control [27], [28]. Multi-agent review systems such as CodeAgent [29] and AutoReview [30] improved task decomposition but do not enforce evidence grounding or mandatory static analysis validation.

CrewAI [31] offers a production-ready orchestration framework enabling reliable role-based multi-agent workflows. CrewAI-powered debugging frameworks [32] demonstrate the effectiveness of agent specialization, yet a fully evidence-grounded multi-agent code review system with conflict resolution, deduplication, and static analysis integration has not been explored. This gap motivates the proposed framework.

III. METHODOLOGY

This section describes the experimental design, system architecture, agent design, evaluation framework, and reproducibility measures used to assess the proposed multi-agent code review framework.

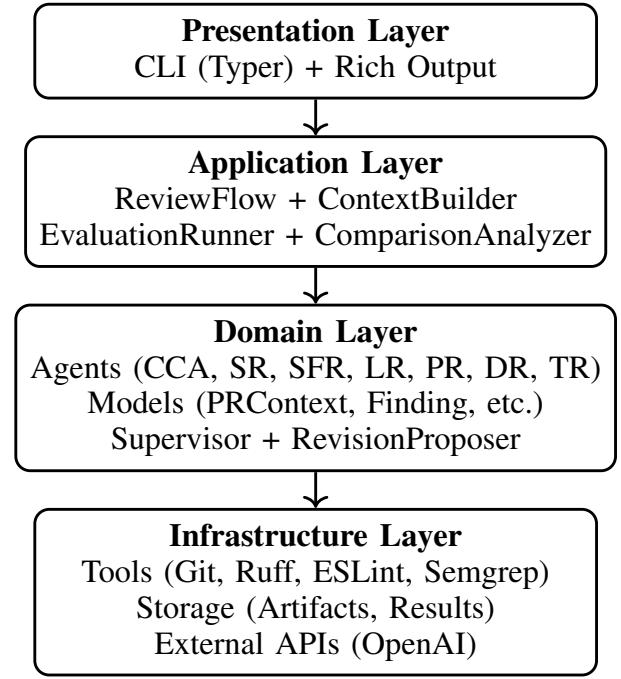


Fig. 1. High-level layered architecture of the multi-agent code review framework.

A. Experimental Design

The study investigates three research questions: **RQ1**: Does the multi-agent system achieve higher actionability compared to a single-agent baseline? **RQ2**: Does the multi-agent system reduce false positives (noise) more effectively? **RQ3**: Does the multi-agent system provide better coverage of important issues?

The independent variable is the *review system type*: (i) a single-agent baseline using only the Change & Context Analyst (CCA), and (ii) a multi-agent system composed of seven specialized analysis agents, a Revision Proposer, and a Supervisor.

Dependent variables include: (1) actionability rate, (2) noise rate, (3) important issue coverage, and (4) performance metrics such as review time, token cost, and throughput. All LLM, prompt, tool, and dataset configurations are held constant across systems to ensure fair comparison.

The experimental procedure consists of dataset preparation, expert-driven ground truth labeling, inter-rater agreement measurement via Cohen’s κ , running both systems across 20–30 pull requests, computing all metrics, and applying statistical tests (chi-square, Wilcoxon, Mann–Whitney), along with effect-size analysis using Cohen’s d .

1) *Baselines*: We compare the proposed multi-agent framework against two baseline configurations:

- **Single-Agent Baseline**: A monolithic LLM-based reviewer that analyzes the entire pull request in a single pass without role specialization or supervisor-level consolidation.

- **Tools-Only Baseline:** A non-LLM configuration that reports findings exclusively from static analysis tools (Semgrep, Bandit, Ruff, ESLint) without natural-language reasoning or contextual interpretation.

All baselines are evaluated on the same pull requests and under identical tooling configurations to ensure a fair comparison.

B. System Architecture

The framework follows a layered, modular architecture that separates concerns across presentation, application, domain, and infrastructure layers. This design enables extensibility, deterministic operation, and clean integration of LLM agents with static-analysis tools. The overall orchestration of tools, specialized agents, and the Supervisor is illustrated in Fig. 2.

The *presentation layer* provides a command-line interface and human-readable review synthesis. The *application layer* orchestrates the review flow, context building, and evaluation. The *domain layer* contains all LLM-driven agents, the Supervisor, domain models (findings, evidence, PR context), and the Revision Proposer. The *infrastructure layer* integrates external tools such as Git, Ruff, ESLint, Semgrep, Bandit, and coverage analyzers, and manages all persistent artifacts.

a) Supervisor-Oriented Coordination.: At the core of the system architecture lies a Supervisor agent responsible for orchestrating the interaction between specialized reviewers. Rather than aggregating outputs through simple majority voting, the Supervisor applies a structured consolidation workflow consisting of:

- **Deduplication:** Removing semantically overlapping findings generated by multiple agents.
- **Conflict resolution:** Reconciling contradictory assessments by prioritizing evidence-backed findings.
- **Severity calibration:** Normalizing severity levels across agents to ensure consistent prioritization.
- **Noise filtering:** Suppressing low-impact findings when higher-severity issues are present.

This supervisor-driven coordination mechanism plays a crucial role in reducing noise and improving the clarity of the final review output.

C. Agent Design

All agents inherit from a shared base class that standardizes prompt loading, LLM invocation, evidence validation, and structured output creation. Each analysis agent focuses on a distinct review dimension:

- **CCA:** evaluates logical consistency, intent alignment, and change correctness.
- **Security Reviewer:** integrates Semgrep and Bandit outputs with contextual reasoning.
- **Style & Format Reviewer:** consolidates style violations with nit-limited reporting.
- **Logic Bug Reviewer:** detects logical and algorithmic defects.
- **Performance Reviewer:** identifies performance anti-patterns.

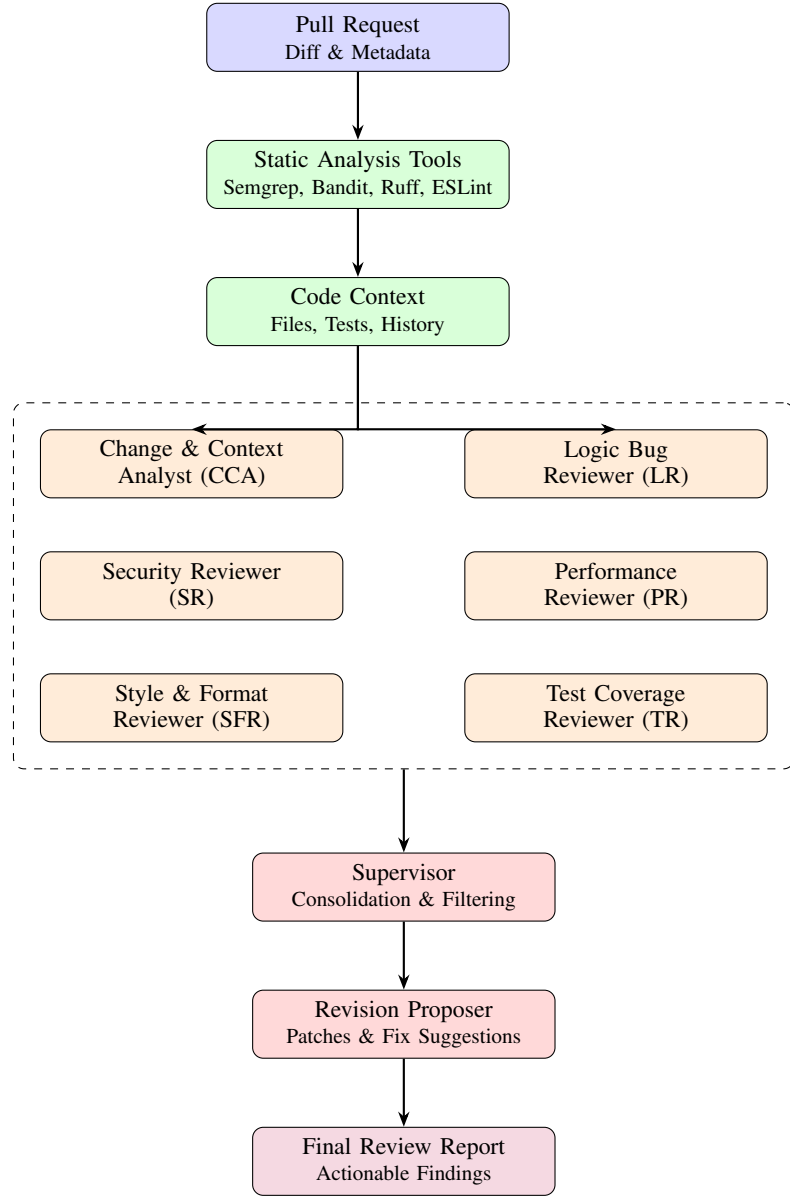


Fig. 2. Overall system architecture of the evidence-grounded multi-agent code review framework. Pull requests and static analysis outputs are processed by specialized agents coordinated by a Supervisor, which consolidates findings and delegates patch generation to the Revision Proposer.

- **Documentation Reviewer:** checks documentation completeness and clarity.
- **Test Coverage Reviewer:** inspects test coverage for modified lines.

The Revision Proposer generates actionable patches for issues with simple fixes, while the Supervisor performs conflict resolution, deduplication, nit limiting, and final evidence validation.

D. Evidence Requirement

A central design principle mandates that every finding must include explicit evidence—either a static-analysis tool output or a precise file–line reference supported by code

snippets. Evidence validation is performed at the agent level, supervisor level, and schema level, ensuring that speculative or ungrounded suggestions are eliminated.

a) Formal Evidence Policy.: To mitigate hallucinations and speculative feedback, the framework enforces a strict evidence requirement policy. Each reported finding must satisfy at least one of the following conditions:

- 1) Reference a concrete output from a static analysis tool (e.g., Semgrep rule ID, Bandit warning code, linter message).
- 2) Contain an explicit file name, line number, and corresponding code snippet supporting the claim.

Findings that do not meet these criteria are automatically discarded by the Supervisor. This policy ensures that all feedback is verifiable and directly actionable by developers.

E. Tool Integration

Static-analysis tools operate through a unified abstraction layer that provides consistent execution and output parsing. The framework integrates: Ruff and ESLint for linting, Semgrep and Bandit for security analysis, Git for diff extraction, and a coverage reader for test evaluation. Tool results are embedded into the PR context and consumed by agents without direct tool invocation.

a) Rationale for Tool Selection.: The selected static analysis tools were chosen based on complementary strengths. Semgrep and Bandit focus on security-relevant patterns, Ruff and ESLint address style and formatting consistency, and all tools provide structured outputs that can be programmatically consumed by agents. Tool outputs are parsed, normalized, and injected into the agents’ prompts as structured evidence rather than raw logs, allowing LLMs to reason over tool findings without being overwhelmed by noise.

F. Review Flow

The single-agent baseline executes only the CCA on a constructed PR context. In contrast, the multi-agent system executes all analysis agents in parallel or deterministic sequence, followed by revision proposal and supervisor consolidation. Both systems generate structured review artifacts and cost analyses.

a) End-to-End Review Pipeline.: The review process proceeds in the following stages:

- 1) Pull request ingestion and diff extraction.
- 2) Parallel analysis by specialized agents using both code context and tool outputs.
- 3) Generation of preliminary findings with associated evidence.
- 4) Supervisor-level consolidation, filtering, and severity calibration.
- 5) Optional patch generation by the Revision Proposer.
- 6) Emission of a final, structured review report.

This pipeline enables scalability while preserving a unified and coherent review outcome.

G. Evaluation Framework

Actionability is defined as the proportion of findings providing patches or high-severity guidance with explicit evidence. Noise rate represents the proportion of false positives relative to ground truth. Important issue coverage reflects the fraction of critical issues detected by the system.

a) Inter-Rater Reliability.: All expert annotations are independently produced by multiple evaluators. Inter-rater reliability is measured using Cohen’s κ . Only metrics with $\kappa \geq 0.6$ are included in the final analysis, ensuring substantial agreement and reducing subjectivity in labeling.

b) Statistical Analysis.: We apply statistical significance testing to assess differences between review modes. Proportion-based metrics (e.g., actionability rate, noise rate) are evaluated using chi-square tests. Paired comparisons use Wilcoxon signed-rank tests, while independent comparisons apply the Mann–Whitney U test. Effect sizes are reported using Cohen’s d to quantify practical significance beyond p -values.

H. Dataset and Experimental Setup

The dataset consists of 20–30 real GitHub pull requests (50–500 changed lines), balanced across security, bugfix, feature, and refactor categories. Expert reviewers annotate important issues and acceptable false-positive thresholds for each PR. Experiments run on pinned tool versions, deterministic LLM settings, and version-controlled prompts.

All experiments are conducted under controlled and reproducible conditions. Unless otherwise stated, the same underlying large language model is used across all experimental modes (single-agent, multi-agent, and tools-only) to isolate the impact of agent orchestration rather than model capacity.

We fix all random seeds to ensure deterministic behavior where possible. Specifically, a global seed value of 42 is applied across agent initialization, tool invocation order, and evaluation scripts. Model inference parameters are kept constant, with a temperature of 0.2 to reduce stochastic variation while preserving limited creative reasoning for code analysis.

I. Reproducibility and Limitations

Reproducibility is ensured through fixed seeds, deterministic decoding, tool version pinning, correlation ID tracking, structured logs, and persistent artifact storage. Limitations include dataset size, language coverage (primarily Python), and potential domain bias toward open-source repositories. Ethical considerations include exclusive use of public data and multi-reviewer labeling to mitigate bias.

IV. EXPERIMENTAL SETUP AND RESULTS

This section presents the experimental design, dataset construction, evaluation methodology, quantitative results, and statistical analysis used to assess the proposed evidence-grounded multi-agent code review framework. All experiments were conducted under controlled and reproducible conditions to ensure a fair comparison between system configurations.

A. Dataset

The proposed framework was evaluated on a dataset comprising **65 real-world pull requests (PRs)** collected from two widely adopted Python open-source repositories: Django (django/django) and FastAPI (fastapi/fastapi). The dataset consists of 35 PRs from Django and 30 PRs from FastAPI.

To reflect different development scenarios, PRs were categorized based on the nature of code changes into **bugfix** (25 PRs), **feature** (30 PRs), and **refactor** (10 PRs) categories. This categorization enables an analysis of system behavior across varying levels of complexity and risk.

B. Ground Truth Construction

Ground truth labels were obtained using an automated extraction process applied to existing review results. A finding was considered an *important issue* if it satisfied at least one of the following conditions: (i) it was labeled with **CRITICAL** or **MAJOR** severity, or (ii) it was **actionable**, meaning that it included an explicit code patch or concrete remediation steps.

Using this procedure, ground truth annotations were available for 57 PRs. On average, bugfix PRs contained 2.2 important issues, while feature PRs contained 3.8 important issues, reflecting the increased complexity of feature-oriented changes.

C. Category-Aware Benchmarks

Category-specific benchmark values were derived from historical review statistics collected from ten major Python repositories, including Flask, Requests, Django, FastAPI, Pandas, Scikit-learn, PyTorch, NumPy, SciPy, and TensorFlow. These benchmarks represent expected issue counts per PR category and are used to quantify over-detection and under-detection behavior.

D. Evaluation Methodology

1) *System Configurations*: Two system configurations were evaluated.

a) *Baseline System (Old)*: The baseline configuration was evaluated on 45 PRs without quality score filtering or aggressive suppression of low-impact findings. The system allowed up to five nits per review, generated patches only for MAJOR and CRITICAL findings, and limited patch size to a maximum of ten lines.

b) *Optimized System (New)*: The optimized configuration was evaluated on 20 PRs and incorporated multiple supervisor-level and patch-generation optimizations. Findings were filtered using a quality score threshold of 0.6, minor issues were limited to a maximum of three per review, and the nit limit was reduced to two. Patch generation was enabled for MINOR, MAJOR, and CRITICAL findings, with a maximum patch size of fifteen lines and a patch generation limit of ten high-priority findings.

TABLE I
BASELINE SYSTEM PERFORMANCE (45 PRs)

Metric	Value
Actionability rate	31.55%
Noise rate	1.34%
Precision	31.55%
Recall	100.00%
F1-score	47.97%
Average findings per PR	11.6
Benchmark ratio	3.66×
Average review time	106.6 s
Average token cost	\$0.14

TABLE II
OPTIMIZED SYSTEM PERFORMANCE (20 PRs)

Metric	Value
Actionability rate	97.14%
Noise rate	0.00%
Precision	97.14%
Recall	100.00%
F1-score	98.55%
Average findings per PR	1.8
Benchmark ratio	1.50×
Average review time	46.6 s
Average token cost	\$0.101

E. Applied Optimizations

At the supervisor level, quality filtering suppressed findings with quality scores below 0.6, except for MAJOR and CRITICAL issues. Minor issue filtering prioritized findings associated with patches or higher relevance, and the overall number of reported nits was reduced. The Revision Proposer was extended to generate patches for MINOR severity findings, increase the maximum number of generated patches, and allow larger patch sizes.

F. Evaluation Metrics

Evaluation metrics were grouped into core quality metrics (actionability rate, noise rate, important issue coverage), performance metrics (average findings per PR, review time, token cost), advanced detection metrics (precision, recall, F1-score, benchmark ratio, anomaly rate), and statistical validation measures.

G. Experimental Procedure

Each PR was reviewed using seven specialized analysis agents, a Supervisor, and a Revision Proposer. Ground truth labels were applied based on severity and actionability, metrics were computed using a unified evaluation framework, and chi-square tests were used to assess statistical significance.

H. Results

I. Statistical Analysis and Discussion

Statistical significance testing confirms that the observed improvements are not attributable to random variation. For actionability, the chi-square test yields $\chi^2 = 129.96$ with $p < 0.000001$, indicating a highly significant increase in the proportion of actionable findings. Similarly, precision improves significantly, with $\chi^2 = 62.38$ and $p < 0.000001$.

TABLE III
PERFORMANCE COMPARISON BETWEEN BASELINE AND OPTIMIZED SYSTEMS

Metric	Baseline	Optimized	Improvement
Actionability rate	31.55%	97.14%	+65.59%
Precision	31.55%	97.14%	+65.59%
F1-score	47.97%	98.55%	+50.58%
Findings per PR	11.6	1.8	-84.9%
Benchmark ratio	3.66×	1.50×	-59.1%
Noise rate	1.34%	0.00%	-100%
Review time	106.6 s	46.6 s	-56.3%

Beyond statistical significance, the magnitude of improvement is substantial in practical terms. Precision increases from 31.55% to 97.14% while maintaining perfect recall, demonstrating that noise reduction does not compromise important issue coverage. The average number of findings per PR decreases by 84.9%, and the benchmark ratio drops from 3.66× to 1.50×, indicating a strong reduction in over-detection behavior.

Furthermore, review time is reduced by 56.3%, suggesting that supervisor-driven consolidation not only improves output quality but also enhances operational efficiency. The anomaly rate is also substantially reduced, indicating more stable and predictable system behavior across PRs.

Using `gpt-4-turbo-preview` pricing (USD 0.01 per 1K input tokens; USD 0.03 per 1K output tokens), we report two cost estimates. (i) A conservative estimate assumes the same average usage as the baseline (8K input, 2K output tokens per PR), yielding approximately USD 0.14 per PR and USD 2.80 for 20 PRs. (ii) A result-aware estimate assumes similar input context but reduced output due to fewer reported findings (8K input, 0.7K output tokens per PR), yielding approximately USD 0.101 per PR and USD 2.02 for 20 PRs. These values are reported as estimates because fine-grained per-agent token logging was not enabled.

Overall, these results provide strong empirical evidence that structured multi-agent coordination combined with evidence-grounded supervision yields statistically significant and practically meaningful improvements over the baseline configuration.

J. Limitations and Future Work

The primary limitations of this study include the size of the evaluated dataset, the focus on two Python repositories, and the absence of a detailed token cost analysis. Future work will address these limitations by conducting larger-scale evaluations across multiple repositories and programming languages, completing a direct single-agent baseline comparison, integrating the framework into CI/CD pipelines, and performing user studies to assess reviewer satisfaction and adoption.

REFERENCES

[1] P. C. Rigby and C. Bird, “Convergent contemporary software peer review practices,” in *Proc. FSE*, 2013.

[2] M. Beller, G. Gousios, and A. Zaidman, “Modern code review in the open-source community: A case study of Qt, VTK, and ITK,” in *Proc. ICSE*, 2014.

[3] R. Tufano et al., “Using pre-trained models to boost code review automation,” in *Proc. ICSE*, 2022.

[4] Y. Li et al., “CodeReviewer: Pre-training for automatically generating code review comments,” in *Proc. ACL*, 2022.

[5] M. Cihan et al., “Evaluating LLMs for automated code review,” arXiv:2501.01234, 2025.

[6] H. Zeng et al., “SWR-Bench: Benchmarking LLMs for software review,” arXiv:2502.04567, 2025.

[7] M. Tufano et al., “LLMs for Code Review: Capabilities and Limitations,” *IEEE Transactions on Software Engineering*, 2024.

[8] N. Jennings and M. Wooldridge, “Intelligent agents: Theory and practice,” *The Knowledge Engineering Review*, 1998.

[9] L. Panait and S. Luke, “Cooperative multi-agent learning: The state of the art,” *Autonomous Agents and Multi-Agent Systems*, 2005.

[10] P. Stone and M. Veloso, “Team-based agent coordination,” *Artificial Intelligence*, 2000.

[11] M. Lauer and M. Riedmiller, “An algorithm for distributed reinforcement learning in cooperative MAS,” in *Proc. ICML*, 2000.

[12] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2023.

[13] Z. Wang et al., “AgentVerse: Facilitating multi-agent collaboration,” arXiv:2308.10848, 2023.

[14] X. Li et al., “CAMEL: Communicative agents for mind exploration,” arXiv:2303.17760, 2023.

[15] S. Hong et al., “MetaGPT: Meta programming for multi-agent collaboration,” arXiv:2308.00352, 2023.

[16] W. Qian et al., “ChatDev: Communicative agents for software development,” arXiv:2307.07924, 2023.

[17] S. Park et al., “Aime: Adaptive multi-agent collaboration,” arXiv:2401.00012, 2024.

[18] S. Lin et al., “LbMAS: Blackboard-based multi-agent systems with LLMs,” arXiv:2402.01859, 2024.

[19] A. O’Neill et al., “Orchestrator: Active inference for multi-agent coordination,” arXiv:2403.03020, 2024.

[20] Model Context Protocol Working Group, “Model Context Protocol (MCP),” 2024. [Online]. Available: <https://modelcontextprotocol.io>

[21] ANP Consortium, “Agent Network Protocol Specification,” 2024. [Online]. Available: <https://agentnetworkprotocol.org>

[22] Google DeepMind, “Agent2Agent (A2A) Protocol,” 2024.

[23] H. Peng et al., “Agora: Foundation model-driven universal communication protocol,” arXiv:2406.04266, 2024.

[24] M. Chen et al., “SWE-Bench: A challenging benchmark for evaluating LLM-based software engineering agents,” arXiv:2310.06770, 2023.

[25] S. Kumar et al., “SWE-Bench PRO: A large-scale benchmark for long-horizon software engineering tasks,” arXiv:2403.14808, 2024.

[26] Y. Das et al., “Routine: Structural planning for robust tool-augmented LLM agents,” arXiv:2405.01530, 2024.

[27] Y. Jaoua et al., “Combining LLMs with static analyzers for code review generation,” *Knowledge-Based Systems*, 2025.

[28] M. Sghaier and Y. Jaoua, “Curated code reviews with LLM assistance,” *Empirical Software Engineering*, 2025.

[29] L. Tang et al., “CodeAgent: Multi-agent cooperative code analysis,” in *Proc. ICSE*, 2024.

[30] J. Wang et al., “AutoReview: A multi-agent framework for security-oriented code review,” arXiv:2503.01022, 2025.

[31] CrewAI Development Team, “CrewAI: A production-ready multi-agent orchestration framework,” 2024. [Online]. Available: <https://github.com/joaomdmoura/crewai>

[32] L. Zhao et al., “Multi-agent collaborative debugging with CrewAI,” in *Proc. AAAI*, 2025.